

HPC Project Report

Leonardo Antonio Riola, Davide Donati, Giacomo Girotto

1 Introduction (??)

2 Assignment 1

2.1 Introduction and objectives

The objective of this project is to implement a simulator that models the propagation of a damped wave. The general partial differential equation describing the phenomenon in space as time varies is the following:

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u \quad (1)$$

Where $u(x, y, t)$ represents the height of the wave on a two-dimensional plane over time, γ is the wave damping coefficient, and c is the wave propagation speed. The simulator calculates the wave height for each point in the coordinate space (x, y) as time t changes, with the ultimate goal of generating an MP4 video of the simulation.

2.2 Implementation

To perform the simulation, it is necessary to convert the continuous model into a discrete model suitable for implementation on a computer. For this purpose, a grid is introduced: space is divided into equally spaced points along the x and y directions with steps Δx and Δy , and time is discretised into instants separated by a time step Δt . Consequently, each point in space is identified by integer indices (i, j) , and time by an index n . The continuous function $u(x, y, t)$ is replaced by a matrix of values $u_{i,j}^n$, representing the value of the function at the grid point (i, j) at time step n . The derivatives appearing in the differential equation are approximated using finite differences. In particular, both central and forward difference methods are implemented in the code and can be tested separately.

3 Assignment 2

3.1 Introduction

Following the initial wave simulator developed with OpenMP, this second assignment extends the application to execute multiple concurrent simulations on an HPC system using a hybrid MPI and OpenMP programming model. The goal is to study multi-wave interaction and interference dynamics under different physical conditions. While OpenMP continues to handle data parallelism by accelerating the 2D stencil computations across local CPU cores, MPI is introduced to manage task parallelism. Specifically, the system executes three distinct simulation scenarios simultaneously within a single execution run:

- Simulation 1: Reproduces the baseline single-wave scenario initiated by a single central impulse at $t = 0$.
- Simulation 2: Models wave superposition and interference phenomena by initiating two simultaneous impulses at different spatial locations at $t = 0$.
- Simulation 3: Introduces time-delayed interference dynamics by generating a second impulse at a spatial location after a predetermined time-shift ($t = t_{start}$). By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

3.2 Implementation

Building upon the code developed in Assignment 1, the implementation extends the simulator to handle multiple concurrent runs. The core numerical scheme for the time-stepping propagation and the mapping function converting field values into grayscale images (**save_pgm**) remain identical to the first version, ensuring consistency in wave physics and visualization. The execution starts by initializing the MPI environment with **MPI_Init**, retrieving the total number of processes with **MPI_Comm_size**, and identifying the rank ID of each process with **MPI_Comm_rank**. To run the three required simulations simultaneously, the program verifies that at least 3 processes are allocated. Task parallelism is managed through conditional branching based on the process rank, where each rank executes a specific scenario and saves its frames into a dedicated directory:

- **Rank 0** (*sim1*): Handles the baseline single-wave simulation initiated by a central impulse of **-84.0f** at coordinates $(M/2, M/2) = (200, 200)$ at $t = 0$.
- **Rank 1** (*sim2*): Manages the simultaneous two-wave interference scenario, applying the central impulse **-84.0f** at $(200, 200)$ alongside a second impulse of **36.0f** at coordinates $(3M/4, M/3) = (300, 133)$ at $t = 0$.
- **Rank 2** (*sim3*): Manages the time-delayed scenario, applying the central impulse **-84.0f** at $(200, 200)$ at $t = 0$ and scheduling a second impulse of **60.0f** at coordinates $(3M/4, 2M/3) = (300, 266)$ to be injected later at time step $n_{\text{start}} = N/7$.

Each MPI process independently allocates memory using **malloc** for its local grid arrays (**u_prev**, **u_curr**, and **u_next**). While data parallelism within each process is still handled by OpenMP as established in Assignment 1, the time-stepping loop incorporates rank-specific logic. Specifically, the check for **rank == 2** at **n == n_start** is placed inside a **pragma omp single** directive within the time loop. This ensures that the time-delayed second impulse for Simulation 3 is injected into the grid at the exact required time step by only one thread per process, avoiding race conditions and preventing multiple threads from applying the impulse redundantly. The same **pragma omp single** block also safely manages frame saving with **save_pgm** and array pointer swapping without thread interference.

The job execution on the HPC cluster is controlled via a SLURM batch script. The configuration requests 3 MPI tasks (*ntasks=3*) to match the 3 simulation ranks, and reserves 4 CPU cores per task (*cpus-per-task=4*). By setting **export OMP_NUM_THREADS** dynamically to the allocated CPUs per task, each MPI process spawns four OpenMP threads, effectively utilizing a total of 12 CPU cores in parallel across the compute node.

3.3 Test and optimizaion

4 Assignment 3